

Recursive Self-Improvement Is Bounded by Decidability, Not Intelligence

A Computability-Theoretic Foundation for the Limits of Self-Improving Systems

Macharia Bari

Abstract

The central question in the recursive self-improvement (RSI) debate has been *how fast* a sufficiently capable system could improve itself. We show that this question is downstream of a prior one that has been overlooked: *can it happen at all?* We formalize RSI as a two-stage operator—generation and verification—over a space of computable systems, and prove that its convergence is governed not by capability, compute, or algorithmic efficiency, but by the *decidability* of the self-verification predicate. For any sufficiently expressive system, the predicate “candidate M' improves on M ” is undecidable by Rice’s theorem, and any verifier that is both sound and complete on general intelligence computes a solution to the halting problem. We introduce a three-regime taxonomy—*tractable* (**P**), *hard* (**NP**), and *undecidable*—and prove that general intelligence RSI lies in the third regime by default. We characterize three escape routes (domain restriction, sound-incomplete verification, external grounding) and prove that each bounds RSI in a distinct way. Hallucination is recovered as the canonical failure mode of an unsound verifier in the undecidable regime. This does not mean RSI is impossible; it means RSI is possible exactly where verification is decidable, and impossible where it is not. The result inverts the dominant framing: RSI is not a capability to be scaled but a structure to be satisfied—the same structure Gödel, Turing, and Rice identified. Intelligence is a quantity; self-improvement is a structure; no quantity of intelligence changes a structure.

Contents

1	Introduction	2
1.1	The Inversion	2
1.2	Why This Is Surprising	3
1.3	Contributions	3
1.4	Roadmap	4
2	Formal Setup	4
2.1	Capability Space	4
2.2	The RSI Operator	5
2.3	The Ideal Verifier	5
2.4	Self-Reference	5
3	The Decidability Barrier	5
3.1	Rice’s Theorem Applied	5
3.2	Halting-Degree of Improvement	6

3.3	The Fixed-Point Problem	7
4	The Three-Regime Taxonomy	7
4.1	Definitions	7
4.2	Main Theorem	8
4.3	The Hierarchy of Barriers	8
5	Hallucination as Unsound Verification	8
5.1	False Positives Compound	8
5.2	Hallucination Defined	9
5.3	The Asymmetry Principle	9
6	The Three Escape Routes	10
6.1	Domain Restriction	10
6.2	Sound-Incomplete Verification	10
6.3	External Grounding	11
6.4	What Counts as an Oracle	11
7	The Gödel–Turing–Rice Lineage	12
8	Domain Taxonomy	12
9	Implications	13
9.1	For Fast Takeoff	13
9.2	For Hallucination	13
9.3	For AI Safety	13
9.4	For Alignment	13
10	Open Problems	14
11	Conclusion	14
A	Notation	15
B	Relation to Prior Work	15
C	Falsifiability	15

1 Introduction

1.1 The Inversion

The recursive self-improvement hypothesis holds that a sufficiently capable system can improve its own improvement mechanism, producing compounding capability gains. The debate around it has been almost entirely about *rate*: fast takeoff versus slow takeoff, intelligence explosion versus diminishing returns, compounding versus plateauing. Both optimists and skeptics have argued within a shared assumption—that RSI is a matter of *how much* intelligence is brought to bear on the problem.

This paper argues that the shared assumption is wrong. RSI is not a quantity question. It is a

decidability question. And decidability is not a resource that more intelligence can purchase. It is a structural property of the problem domain. No amount of capability, compute, or cleverness moves a system from undecidable to decidable, any more than more computing power lets a Turing machine decide its own halting problem.

The central claim:

RSI is effective iff self-verification is decidable in the domain of interest; tractable iff it lies in $\mathbf{P}$; and bounded otherwise. General intelligence RSI is undecidable by default.

The surprise is not that RSI is hard. The surprise is that hardness was never the right frame.

1.2 Why This Is Surprising

The intuition that RSI is a capability problem is nearly irresistible, and it deserves to be stated explicitly so the inversion can be seen for what it is. The reasoning runs:

Intelligence is the ability to solve problems. Self-improvement is a problem. Therefore, a sufficiently intelligent system can solve it. Therefore, RSI is a matter of *how much* intelligence, not *whether*.

Every popular framing—intelligence explosion, takeoff, singularity—inherits this reasoning. It treats RSI as a scaling question. The result of this paper inverts the framing: the thing everyone thought was a matter of degree turns out to be a matter of kind.

This is not a novel shape of surprise. It is the same shape as three of the deepest results of the twentieth century:

- **Gödel (1931).** “A sufficiently powerful formal system can prove all mathematical truths.” False—and not for lack of power.
- **Turing (1936).** “A sufficiently clever machine can decide any well-posed question.” False—and not for lack of cleverness.
- **Rice (1953).** “A sufficiently capable program can analyze programs.” False—and not for lack of capability.

RSI belongs to this lineage. It is the fourth member of a family that mathematics already told us exists. The field has been re-deriving a puzzle that was solved—in the negative—before the field existed.

1.3 Contributions

- (1) A formal model of RSI as a pair of operators $(\mathcal{G}, \mathcal{V})$ over a capability space, with explicit assumptions stated as such.
- (2) A Rice-style reduction showing that general self-verification is undecidable, and a halting-degree theorem showing that any sound and complete general verifier computes a solution to the halting problem.
- (3) A three-regime taxonomy with theorems characterizing RSI behavior in each.
- (4) A characterization of hallucination as the signature of unsound verification in the undecidable regime, including a proof that false positives compound while false negatives stall.

- (5) A proof that the three escape routes—domain restriction, sound-incomplete verification, and external grounding—each bound RSI in a distinct and necessary way.
- (6) A reframing of the fast-takeoff hypothesis as a computability conjecture, and of grounding as a computability-theoretic necessity rather than a normative preference.

1.4 Roadmap

Section 2 sets up the formalism. Section 3 proves the decidability barrier: Rice’s theorem applied to self-verification, and the halting-degree of sound RSI. Section 4 introduces the three-regime taxonomy and the main theorem. Section 5 recovers hallucination as the failure mode of unsound verification. Section 6 analyzes the three escape routes. Section 7 places the result in the Gödel–Turing–Rice lineage. Section 8 gives the domain taxonomy. Section 9 draws implications. Section 10 lists open problems. Section 11 concludes.

2 Formal Setup

2.1 Capability Space

Let $\mathcal{M}$ be a class of computable systems (programs, models, agents). For $M \in \mathcal{M}$, let $\text{cap}(M) \in \mathbb{R}$ be a capability measure. Define a partial order $\preceq$ on $\mathcal{M}$ by

$$M \preceq M' \iff \text{cap}(M) \leq \text{cap}(M').$$

The argument to follow rests on four assumptions. We state them once, and openly.

Axiom 2.1 (Effective presentation). $\mathcal{M}$ is effectively presented: every $M \in \mathcal{M}$ has a finite description, and the constructions below (pairing, composition, conditional simulation, recursion) are computable uniformly in these descriptions.

Axiom 2.2 (Semantic capability). cap is a *semantic* property: if $M, M' \in \mathcal{M}$ compute the same partial function, then $\text{cap}(M) = \text{cap}(M')$.

Axiom 2.3 (Non-trivial capability). cap is *non-trivial*: the image $\text{cap}(\mathcal{M}) \subseteq \mathbb{R}$ contains at least two values. Equivalently, there exist $M_-, M_+ \in \mathcal{M}$ with $\text{cap}(M_-) < \text{cap}(M_+)$.

Axiom 2.4 (Closure under simulation). $\mathcal{M}$ contains a universal machine: there is $U \in \mathcal{M}$ that simulates every $M \in \mathcal{M}$ on every input.

Axioms 2.2 and 2.3 are the minimal content of “capability.” They hold for any reasonable measure: task performance, predictive accuracy, reasoning depth. Axiom 2.4 is the scope condition that makes the general case general; the restricted classes of Section 6 are exactly the classes obtained by dropping it.

We further assume cap tracks *genuine capability*, not a syntactic proxy. If a system optimizes a decidable proxy—parameter count, loss on a fixed distribution, score on a fixed benchmark—it may improve the proxy while degrading capability. This is the well-known *proxy-capability gap*, and it is a special case of the verifier-unsoundness analyzed in Section 5. The results below apply to systems that optimize capability; systems that optimize proxies are subject to strictly stronger failure modes.

Remark 2.5 (Semantic versus syntactic). Axiom 2.2 is what separates capability from mere book-keeping. Any predicate that can read a machine’s source code—“has more than 10^9 parameters,”

“was trained on more tokens”—is decidable but not semantic, and systems optimizing such predicates fall into the proxy-capability gap. The theorems below require semantic capability; the proxy case is strictly more pessimistic, not less.

2.2 The RSI Operator

An RSI step is

$$M_{t+1} = \mathcal{A}(M_t, \mathcal{G}(M_t), \mathcal{V}(M_t, \mathcal{G}(M_t))),$$

where:

- $\mathcal{G} : \mathcal{M} \rightarrow \mathcal{M}$ is the *generation operator*—it proposes $M' = \mathcal{G}(M_t)$;
- $\mathcal{V} : \mathcal{M} \times \mathcal{M} \rightarrow \{0, 1\}$ is the *verification predicate*—it decides whether M' improves on M_t ;
- $\mathcal{A}$ is the *adoption rule*—it commits to M' iff $\mathcal{V} = 1$.

The *effective RSI step* is

$$\mathcal{R}(M) = \begin{cases} \mathcal{G}(M) & \text{if } \mathcal{V}(M, \mathcal{G}(M)) = 1, \\ M & \text{otherwise.} \end{cases}$$

The RSI process is the orbit $M_0, M_1, M_2, \dots$ under $\mathcal{R}$.

2.3 The Ideal Verifier

The *ideal verifier* is

$$\mathcal{V}^*(M, M') = \begin{cases} 1 & \text{if } M \preceq M', \\ 0 & \text{otherwise.} \end{cases}$$

Any practical $\mathcal{V}$ is a computable approximation of $\mathcal{V}^*$. The gap between them is the subject of this paper.

2.4 Self-Reference

Definition 2.6 (Self-referential RSI). An RSI operator is *self-referential* if $\mathcal{V}$ is computed by a function of M_t itself—that is, $\mathcal{V} = \mathcal{V}(M_t, \cdot)$.

All non-trivial RSI is self-referential in this sense: the system judges its own improvement. The ascription is not decorative. It is what licenses the use of Rice’s theorem in Section 3: self-referential verification is the paradigmatic non-trivial semantic predicate.

3 The Decidability Barrier

3.1 Rice’s Theorem Applied

Theorem 3.1 (Rice’s theorem). *For any non-trivial semantic property P of partial computable functions, the set $\{e : P(\phi_e)\}$ is undecidable.*

Corollary 3.2 (Self-verification is undecidable). *For any capability measure satisfying Axioms 2.2 and 2.3, the predicate*

$$\mathcal{V}^*(M, M') \equiv [\text{cap}(M) \leq \text{cap}(M')]$$

is undecidable for general $M, M' \in \mathcal{M}$.

Proof of Corollary 3.2. Fix a baseline $B \in \mathcal{M}$ and consider the property

$$P_B(M') \equiv [\text{cap}(B) < \text{cap}(M')].$$

By Axiom 2.3 there exist M_-, M_+ with $\text{cap}(M_-) < \text{cap}(M_+)$. Taking $B = M_-$, we have $\neg P_B(M_-)$ and $P_B(M_+)$; hence P_B is non-trivial. By Axiom 2.2, P_B is a semantic property of the function computed by M' . By Theorem 3.1, $\{M' : P_B(M')\}$ is undecidable. But P_B is the restriction of the improvement predicate $\mathcal{V}^*(B, \cdot)$ to a single fixed baseline. Since a decision procedure for the unrestricted predicate $\mathcal{V}^*(\cdot, \cdot)$ would decide P_B by holding its first argument constant, $\mathcal{V}^*$ is undecidable. $\square$

Remark 3.3. Corollary 3.2 is not a statement about efficiency. It is a statement about *existence*. No computable verifier $\mathcal{V}$ can be sound and complete for the general improvement problem, over and above efficiency concerns of any kind.

3.2 Halting-Degree of Improvement

We formalize the computational strength of the self-verification problem directly. Let $K = \{(e, x) : \phi_e(x) \downarrow\}$ denote the halting problem, and write $A \leq_T B$ for Turing reducibility.

Lemma 3.4 (Rice-style reduction). *Under Axioms 2.2, 2.3, and 2.4, there is a total computable map $(e, x) \mapsto (M_{e,x}, M'_{e,x})$ such that*

$$M_{e,x} \preceq M'_{e,x} \iff \phi_e(x) \downarrow,$$

and moreover the comparison is strict on the halting side: $\phi_e(x) \downarrow$ implies $\text{cap}(M_{e,x}) < \text{cap}(M'_{e,x})$.

Proof of Lemma 3.4. By Axiom 2.3 choose f_- and f_+ with $\text{cap}(f_-) = c_- < c_+ = \text{cap}(f_+)$. Let $M_{e,x}$ be a fixed program computing f_- , independent of the pair. By Axiom 2.1, define $M'_{e,x}$ as the program that on input n first simulates $\phi_e(x)$; if the simulation halts, it returns the result of $f_+(n)$; if the simulation diverges, the program diverges on input n . Then the function computed by $M'_{e,x}$ is

$$\phi_{M'_{e,x}} = \begin{cases} f_+ & \text{if } \phi_e(x) \downarrow, \\ \Omega & \text{if } \phi_e(x) \uparrow, \end{cases}$$

where Ω is the everywhere-undefined function. As the capability of a program that diverges on all inputs is at most that of any function (adjoin f_- to dominate it if necessary, which preserves the effective construction), we obtain $\text{cap}(M'_{e,x}) = c_+ > c_-$ exactly when $\phi_e(x)$ halts and $\text{cap}(M'_{e,x}) \leq c_-$ otherwise. The map is computable by Axiom 2.1 (in particular by the universal simulator of Axiom 2.4 used inside the simulation). $\square$

Theorem 3.5 (Improvement is at least as hard as halting). *Let $\mathcal{V}$ be a verifier that is sound ($V(M, M') = 1 \Rightarrow M \preceq M'$) and complete ($M \preceq M' \Rightarrow V(M, M') = 1$) on general $\mathcal{M}$ satisfying Axioms 2.2, 2.3, and 2.4. Then the halting problem reduces to the decision problem of $\mathcal{V}$:*

$$K \leq_T \Pi_{\mathcal{V}}.$$

In particular, no such sound-and-complete $\mathcal{V}$ is computable.

Proof. Given (e, x) , compute $(M_{e,x}, M'_{e,x})$ by Lemma 3.4. Then $M_{e,x} \preceq M'_{e,x}$ iff $\phi_e(x) \downarrow$. Query $\mathcal{V}(M_{e,x}, M'_{e,x})$:

- if $\phi_e(x) \downarrow$, then $M_{e,x} \preceq M'_{e,x}$; completeness forces $\mathcal{V} = 1$;
- if $\phi_e(x) \uparrow$, then $M_{e,x} \not\preceq M'_{e,x}$; soundness forces $\mathcal{V} = 0$ (a 1 would assert a false improvement).

Hence $\mathcal{V}(M_{e,x}, M'_{e,x}) = 1 \iff (e, x) \in K$, a Turing reduction. Since K is undecidable, $\mathcal{V}$ is not computable. $\square$

Theorem 3.5 sharpens the statement of the abstract: a sound general-purpose verifier—one that never certifies a non-improvement—is *at least* as powerful as a halting oracle, in the sense that if it were complete as well it would decide the halting problem outright.

Corollary 3.6 (No sound-and-complete RSI). *No sound, complete, computable, general-purpose verifier $\mathcal{V}$ exists; equivalently, no sound and complete computable RSI operator over general $\mathcal{M}$ exists.*

Proof. Immediate from Theorem 3.5 and the undecidability of K . $\square$

Remark 3.7 (Scope condition). Theorem 3.5 applies to any $\mathcal{M}$ satisfying Axiom 2.4 and closed under the construction of Lemma 3.4. Restricted classes (finite-state, bounded-depth, decidable fragments) may escape the barrier. These restricted classes are precisely the ones analyzed in Section 6: the escape routes are the restricted classes.

3.3 The Fixed-Point Problem

The fixed point M^* with $\mathcal{R}(M^*) = M^*$ satisfies the self-referential equation

$$\mathcal{V}(M^*, \mathcal{G}(M^*)) = 0,$$

or the trivial identity when $\mathcal{G}(M^*) = M^*$. Determining existence, uniqueness, or reachability of such fixed points is, in the general case, equivalent to solving the halting problem: the orbit under $\mathcal{R}$ stays in place precisely when the verifier is queried on the diagonal, and deciding even the first such query requires Theorem 3.5. This is why the fixed-point problem is not a complexity question but a *computability* question.

4 The Three-Regime Taxonomy

4.1 Definitions

Let $\Pi_{\mathcal{V}}$ denote the decision problem instantiated by $\mathcal{V}$: given (M, M') , decide whether $M \preceq M'$.

Definition 4.1 (Regimes). An RSI domain is:

- *tractable* if $\Pi_{\mathcal{V}} \in \mathbf{P}$;
- *hard* if $\Pi_{\mathcal{V}}$ is **NP**-hard but decidable;
- *undecidable* if $\Pi_{\mathcal{V}}$ is not decidable.

Remark 4.2 (Empirical and asymptotic regimes). The hard regime is where current systems live: open-ended factuality, long-horizon reasoning, and general language modeling are not obviously undecidable—they may be **NP**-hard. The undecidable regime is the asymptotic default for expressive systems. The taxonomy therefore does double duty: it classifies domains *and* explains why current systems exhibit the failure modes they do without yet having reached the full barrier.

4.2 Main Theorem

Theorem 4.3 (Decidability-bounded RSI). *Let $\mathcal{R}$ be an RSI operator with generation $\mathcal{G}$ and verifier $\mathcal{V}$. Then:*

- (1) (**Tractable regime**) *If $\Pi_{\mathcal{V}} \in \mathbf{P}$ and $\mathcal{G}$ is a δ -improver (produces M' with $\text{cap}(M') \geq \text{cap}(M) + \delta$ whenever such M' is reachable), then $\text{cap}(M_t)$ grows at least linearly at rate δ .*
- (2) (**Hard regime**) *If $\Pi_{\mathcal{V}}$ is $\mathbf{NP}$ -hard and $\mathbf{P} \neq \mathbf{NP}$, then any polynomial-time verifier $\mathcal{V}$ is unsound on some instance (M, M') with $M, M' \in \mathcal{M}$. RSI progress is bounded above by the fraction of the improvement space on which $\mathcal{V}$ is sound.*
- (3) (**Undecidable regime**) *If $\Pi_{\mathcal{V}}$ is undecidable, no sound computable $\mathcal{V}$ exists. RSI requires domain restriction, sound-incomplete verification, or an external oracle (see Section 6).*

Proof. (1) By induction. Assume $\text{cap}(M_t) \geq \text{cap}(M_0) + t\delta$. If $\mathcal{V}$ accepts, then $\text{cap}(M_{t+1}) \geq \text{cap}(M_t) + \delta$; if $\mathcal{V}$ rejects, $\text{cap}(M_{t+1}) = \text{cap}(M_t)$. Either way the bound holds, and since $\Pi_{\mathcal{V}} \in \mathbf{P}$ the verifier runs in polynomial time, so the induction is effective.

(2) Sound verification on all of $\mathcal{M}$ is equivalent to deciding $\Pi_{\mathcal{V}}$: a verifier is sound and complete exactly when it solves $\Pi_{\mathcal{V}}$, and sound-only verifiers are the incomplete solutions. If $\Pi_{\mathcal{V}}$ is $\mathbf{NP}$ -hard, any polynomial-time $\mathcal{V}$ fails on some instances; among the failures are false positives (unsoundness on that instance) or false negatives (incompleteness), and by the asymmetry of 5 the binding constraint on progress is the false-positive fraction. Hence progress is bounded by the sound-region measure, which is the fraction $\mu(\text{sound}(\mathcal{V}))$ of the improvement space on which $\mathcal{V}$ agrees with $\mathcal{V}^*$.

(3) By Corollary 3.6, no sound computable $\mathcal{V}$ exists. The remaining behaviors are characterized in Section 6. $\square$

4.3 The Hierarchy of Barriers

$$\text{Undecidability} \succ \mathbf{NP}\text{-hardness} \succ \mathbf{P}.$$

RSI is governed by the *hardest* barrier applicable to the domain. The $\mathbf{P}$ versus $\mathbf{NP}$ question captures the generation-verification asymmetry at the resource level; the halting problem captures it at the level of existence. The latter subsumes the former: a barrier at the level of existence persists across all resource budgets.

Remark 4.4 (Three claims, one barrier). Theorem 4.3(1)–(3) are not three different claims. They are one claim, $\Pi_{\mathcal{V}}$ -dependent: RSI converges exactly where $\Pi_{\mathcal{V}}$ is tractable, degrades where it is intractable, and degenerates where it is undecidable. The “hardness” framing of the takeoff debate is thus a special case, in the combinatorial sense, of a computability-theoretic result.

5 Hallucination as Unsound Verification

5.1 False Positives Compound

Definition 5.1 (False positive and false negative). For a verifier $\mathcal{V}$:

- *False positive*: $\mathcal{V}(M, M') = 1$ but $M \not\preceq M'$.
- *False negative*: $\mathcal{V}(M, M') = 0$ but $M \preceq M'$.

Proposition 5.2 (Asymmetry of errors). *In an RSI loop, false positives compound corruption; false negatives compound at most delay.*

Proof. Let $\text{cap}_t = \text{cap}(M_t)$. A false negative leaves $M_{t+1} = M_t$, so $\text{cap}_{t+1} = \text{cap}_t$: the loss is a missed opportunity, not a corrupted state. A false positive adopts M' with $\text{cap}(M') < \text{cap}_t$, so $\text{cap}_{t+1} < \text{cap}_t$. If the verifier’s bias is systematic (persistent across t), the adoption of a below-baseline system feeds a generation operator that samples the neighborhood of that system, and the argument repeats: cap decreases monotonically along the accepted orbit. Error does not annihilate capability in one step, but it compounds it multiplicatively in the acceptance signal, which is what governs progress. $\square$

Remark 5.3 (Recoverability). The asymmetry is sharp because stalling is recoverable—an improved-policy search can be resumed—while an accepted corruption is not: the system has committed to the corrupted code as its new ground truth, and its own future judgments are downstream of that commitment. This is the formal seed of the safety discussion in Section 9.

5.2 Hallucination Defined

Definition 5.4 (Hallucination rate). Fix a distribution π over candidate pairs (M, M') generated by the model’s own generative process. The *hallucination rate* of $\mathcal{V}$ under π is

$$\text{hal}(\mathcal{V}, \pi) = \Pr_{(M, M') \sim \pi} [\mathcal{V}(M, M') = 1 \text{ and } M \not\preceq M'].$$

Theorem 5.5 (Hallucination correspondence). *In the undecidable regime, every computable verifier is either unsound (nonzero hallucination rate on any distribution with full support on the diagonal) or incomplete (stalling). Hallucination is therefore not incidental to RSI in general domains; it is the canonical failure mode of the only computable verifiers available.*

Proof. By Corollary 3.6, no computable $\mathcal{V}$ is both sound and complete on $\mathcal{M}$. Two cases exhaust the computable possibilities. *Case 1: $\mathcal{V}$ unsound.* There is (M, M') with $\mathcal{V}(M, M') = 1$ and $M \not\preceq M'$. Any distribution π with positive mass on this pair (in particular the full-support diagonal) has $\text{hal}(\mathcal{V}, \pi) > 0$. By Proposition 5.2 the false positives compound. *Case 2: $\mathcal{V}$ sound-but-incomplete.* No false positives occur, so $\text{hal}(\mathcal{V}, \pi) = 0$ on all π ; but some genuine improvement (M, M') with $M \preceq M'$ is rejected, and every RSI orbit through a rejected state stalls indefinitely (Section 6).

There is no third computable case: sound-and-complete is excluded by Corollary 3.6. $\square$

Corollary 5.6 (No fourth option). *In the undecidable regime, an RSI system has exactly two computable behaviors: it stalls (sound-incomplete) or it hallucinates (unsound). The third option—sound and complete—is unavailable by Rice. There is no fourth option.*

5.3 The Asymmetry Principle

Generation and verification are asymmetric in cost:

$$\text{cost}(\mathcal{G}) \not\approx \text{cost}(\mathcal{V}).$$

In open-ended domains, $\mathcal{G}$ produces fluent, coherent candidates cheaply; $\mathcal{V}$ requires grounding the candidate against an external reality. This *generation-verification gap* is the RSI analog of the **P**

versus **NP** gap—and, in the general case, of the halting problem itself. In the tractable regime the asymmetry is mild (verification is cheap: a proof checker, a test suite). In the undecidable regime it is total: the generation side has no fixed counterpart that can certify it.

Observation (Why hallucination is the default direction). Unsoundness is the *easier* failure. A verifier that certifies too much is cheap to write and locally plausible; a verifier that certifies nothing is useless. Tension toward completeness therefore biases every practical verifier toward unsoundness, exactly as Proposition 5.2 predicts is the dangerous direction. The empirically observed prevalence of hallucination over stalling in self-training loops follows from this asymmetry alone.

6 The Three Escape Routes

If general RSI is undecidable, how does any RSI work at all? Three routes, each with a distinct cost. The three routes correspond exactly to the three ways to make $\Pi_{\mathcal{V}}$ decidable: shrink the domain, weaken completeness, or move the decision outside the system.

6.1 Domain Restriction

Definition 6.1 (Restriction). A *restriction* is a subclass $\mathcal{M}_0 \subseteq \mathcal{M}$ on which $\Pi_{\mathcal{V}}$ is decidable.

Theorem 6.2 (Restriction compounds within its box). *For any restriction $\mathcal{M}_0$ on which $\Pi_{\mathcal{V}} \in \mathbf{P}$, RSI compounds within $\mathcal{M}_0$ at rate δ (Theorem 4.3(1)). Outside $\mathcal{M}_0$, no guarantee holds.*

Proof. Within $\mathcal{M}_0$, $\Pi_{\mathcal{V}} \in \mathbf{P}$; restate Theorem 4.3(1) with domain $\mathcal{M}_0$. For the “outside” direction, note that the verifier is posed instances in $\mathcal{M} \setminus \mathcal{M}_0$ it was never designed for; there is nothing to certify. Formally, $\mathcal{V}$ restricted to $\mathcal{M}_0$ decides $\Pi_{\mathcal{V}} \cap (\mathcal{M}_0 \times \mathcal{M}_0)$, which says nothing about $\Pi_{\mathcal{V}} \cap (\mathcal{M} \times \mathcal{M}_0)$. $\square$

Example 6.3 (Boxed RSI successes). Formal proof checking (Lean, Coq), code backed by test suites, and closed games (AlphaZero) are restrictions where verification is decidable and cheap. All produce strong RSI *within their box*. The box is the point: each restricts $\mathcal{M}$ to a class where $\Pi_{\mathcal{V}}$ is **P**-decidable, and the barrier of Theorem 3.5 does not apply.

Corollary 6.4 (No free lunch). *Domain restriction buys decidability at the cost of generality: the set of capabilities reachable within $\mathcal{M}_0$ is a monotone image of $\mathcal{M}_0$ ’s reach, not of $\mathcal{M}$ ’s. There is no free lunch.*

6.2 Sound-Incomplete Verification

Definition 6.5 (Sound-incomplete verifier). A verifier $\mathcal{V}$ is *sound-incomplete* if it is sound but not complete.

Proposition 6.6 (The Gödelian bargain). *A sound-incomplete verifier yields monotone but bounded RSI: progress is guaranteed but cannot exceed the provable-improvement frontier.*

Proof. Soundness prevents false positives, so by Proposition 5.2 capability never decreases along the orbit: $\text{cap}_{t+1} \geq \text{cap}_t$ always. Incompleteness means some genuine improvements (M, M') with $M \preceq M'$ are rejected, so the orbit stalls at the latest state whose successors the verifier accepts. The frontier is the set of improvements provable within the verifier’s formalism: $\{M' :$

$M \preceq M'$ and $\mathcal{V}(M, M') = 1\}$, whose supremum brackets any attainable capability. The ceiling is real: if the frontier is bounded, so is the orbit. $\square$

Corollary 6.7 (Safest, lowest ceiling). *Sound-incomplete verification is the safest RSI regime but has the lowest ceiling. It is the regime to operate in for systems whose failure costs dominate their opportunity costs.*

6.3 External Grounding

Definition 6.8 (Oracle). An *oracle* O is a function outside $\mathcal{M}$ that decides some predicate not computable within $\mathcal{M}$.

Theorem 6.9 (Grounding as computability necessity). *Let $\Pi_{\mathcal{V}}$ be undecidable. Then there exists no sound, computable, self-referential RSI operator. However, for any oracle O with $\Pi_{\mathcal{V}} \leq_T O$, there exists an RSI operator sound relative to O .*

Proof. The negative direction is Corollary 3.6. For the positive direction, let $\mathcal{V}_O$ compute $\Pi_{\mathcal{V}}$ using O as a subroutine: $\mathcal{V}_O(M, M') = 1$ iff $\Pi_{\mathcal{V}}(M, M')$ answers yes. Then $\mathcal{V}_O$ is sound and complete relative to O by construction, and the effective step $\mathcal{R}_O(M) = \mathcal{G}(M)$, if $\mathcal{V}_O(M, \mathcal{G}(M)) = 1$, and M otherwise, is a sound computable-in- O RSI operator. Soundness relative to O means: all acceptances are true improvements relative to $\mathcal{V}^*$, certified by the oracle. $\square$

Corollary 6.10 (Grounding is not optional). *In the undecidable regime, RSI requires exactly one of: domain restriction, sound-incomplete verification, or an external oracle. Grounding is a computability-theoretic necessity, not a heuristic preference.*

6.4 What Counts as an Oracle

The theorem requires that the oracle be *external* to $\mathcal{M}$. In practice, “the environment” is part of the system’s causal history, so the reader may reasonably ask whether the environment is really an oracle or just more computation.

The distinction is computational, not metaphysical. An oracle is any source of truth that decides a predicate not in the system’s decidable fragment. This includes:

- **The physical environment.** Sensor readings decide facts about the world that the system cannot derive from its own computation.
- **Human feedback.** A human decides whether an output is acceptable, and the human’s judgment is not a function of the system’s weights.
- **Formal proof systems.** A proof checker decides whether a proof is valid, and the checker is a decidable procedure external to the model being verified.
- **Retrieval from a corpus.** A document decides a fact, and the document is not generated by the system.

What these have in common is not a metaphysics of externality but a computability-theoretic property: each decides a predicate outside the system’s decidable fragment. The oracle is not “outside” in a spatial sense; it is outside in a *decidability* sense.

Remark 6.11 (Relabeling is not grounding). This is why the escape route is not a loophole. A system cannot become its own oracle by relabeling its internal states as “environment.” By Theorem 3.5, whatever internal subroutine the system consults is just more computation of the same class $\mathcal{M}$; the oracle must decide a predicate that no program in $\mathcal{M}$ decides. By Rice’s theorem the improvement predicate is such a predicate. The oracle must therefore be something the system does not compute—and this is the precise sense in which grounding is necessary, not conventional.

7 The Gödel–Turing–Rice Lineage

The result of this paper is best understood not as a novel impossibility, but as the fourth member of a family that mathematics identified in the twentieth century. Three classical results bound self-improvement:

1. **Gödel’s incompleteness.** A sufficiently expressive system cannot prove all truths about itself. $\Rightarrow$ RSI cannot verify all its own improvements.
2. **Turing’s halting problem.** A sufficiently expressive system cannot decide all facts about its own computation. $\Rightarrow$ RSI cannot decide whether its verifier will halt.
3. **Rice’s theorem.** No non-trivial semantic property of programs is decidable. $\Rightarrow$ “Is this an improvement?” is undecidable in general.

These are not three obstacles. They are *one* obstacle—self-reference breaks decidability—viewed through three lenses. RSI is the engineering attempt to do what these theorems say cannot be done in general. The theorems do not say it is impossible always; they say it is impossible *in general*. The entire game of practical RSI is finding domains where the general impossibility does not bind—and Section 6 identifies the three canonical ways.

The lineage matters because it reframes the debate. The field has treated RSI as a novel puzzle specific to AI. It is not. It is a puzzle about self-reference, and self-reference has been the central theme of mathematical logic since 1931. The AI field has been re-deriving, in engineering language, what Gödel, Turing, and Rice already established in mathematical language: *systems cannot fully decide themselves*.

8 Domain Taxonomy

Domain	Π_V class	RSI outlook	Grounding
Formal proof	P	Strong	Proof checker
Code	P	Strong, test-bounded	Test suite
Closed games	P	Strong	Win/lose
Narrow prediction	P	Strong	Held-out data
Open-ended factuality	Undecidable	Weak; hallucination-prone	External retrieval
“Better AI” (general)	Undecidable	Open problem	Requires oracle

Table 1: Domain taxonomy of RSI by verification complexity.

Observation. RSI succeeds exactly where Π_V is decidable, stalls where it is **NP**-hard, and hallucinates where it is undecidable. This is not coincidence; it is Theorem 4.3 combined with Theorem 5.5.

The escape routes are the interesting part of the taxonomy, not the barrier. RSI works perfectly well in proof checking, code, games, narrow prediction. The honest picture is not “RSI is impossible.” It is: *RSI is possible exactly where you would never have needed to worry about it, and impossible exactly where the takeoff hypothesis lives.* That is a stranger and more specific claim than either the optimists or the pessimists have been making.

9 Implications

9.1 For Fast Takeoff

Fast takeoff requires an unbounded RSI loop. By Theorem 4.3, such a loop requires $\Pi_V \in \mathbf{P}$ *for the general intelligence improvement problem*. By Corollary 3.2, this predicate is undecidable for expressive systems. Therefore:

Corollary 9.1 (Takeoff is a computability conjecture). *Fast takeoff from general RSI is contingent not on a complexity conjecture but on a computability conjecture that contradicts Rice’s theorem in the general case. It requires either a domain restriction so severe it is no longer general, or an oracle not derivable from the system itself.*

The burden of proof is on the takeoff hypothesis. It must exhibit either a general domain in which self-verification is decidable—which would contradict Rice—or an oracle that is not itself the system. Neither is currently available.

9.2 For Hallucination

Hallucination in self-improving systems is not a bug to be patched. It is the *empirical signature* of an unsound verifier in the undecidable regime (Theorem 5.5). Mitigation requires external grounding, not more self-critique. This explains, in a single stroke, why self-critique loops plateau and why retrieval, tool use, and human feedback are not optional but constitutive.

9.3 For AI Safety

A system that cannot soundly verify its own improvement cannot be trusted to self-modify. Safety guarantees for RSI reduce to *verifier soundness guarantees*, which reduce to *grounding guarantees* (Corollary 6.10). The safety question is a computability question.

9.4 For Alignment

Alignment of a self-improving system requires that the system’s verifier be sound with respect to human values. By Corollary 6.10, this requires grounding in something external to the system—human feedback, institutional oversight, or formal specification.

We state the following as a conjecture rather than a theorem, since it depends on modeling alignment as verifier soundness with respect to a value predicate—a stronger assumption than this paper has established:

Conjecture 9.2 (Self-alignment is self-improvement). *If alignment is modeled as verifier soundness with respect to a value predicate V_{human} , and if V_{human} is a non-trivial semantic property, then self-alignment is subject to the same undecidability barrier as self-improvement. A system cannot align itself for the same structural reason it cannot improve itself: it cannot soundly decide the relevant predicate.*

If the conjecture holds, the alignment problem is not merely hard but structurally bounded in the same way the improvement problem is. This would unify two apparently separate research programs under a single computability-theoretic frame.

10 Open Problems

1. **Formalize the self-verification class SV.** What is the complexity class of self-referential verification? How does it relate to **P**, **NP**, **IP**, **MIP**, and the arithmetical hierarchy?
2. **Characterize the provable-improvement frontier.** For a sound-incomplete verifier, what is the shape of the set of provable improvements? Is it generically bounded, and can its bound be computed?
3. **Minimal oracles.** For a given $\mathcal{M}$, what is the weakest oracle O such that RSI is sound and complete relative to O ?
4. **Distributional RSI.** Worst-case undecidability (Theorem 3.5) may not bind on natural distributions. What is the average-case behavior of self-verification?
5. **Approximate verification.** Does partial soundness (correct with probability $> 1/2$) suffice for compounding RSI, or does it inevitably drift?
6. **The fixed-point spectrum.** Characterize the set of self-referential fixed points and their stability under verifier perturbation.
7. **Empirical signatures.** Can the three regimes be distinguished empirically in current systems (e.g., by hallucination rate scaling with self-training generations)?
8. **The alignment conjecture.** Is Conjecture 9.2 true? If so, can the value predicate be characterized in terms of the arithmetical hierarchy?

11 Conclusion

We have argued that recursive self-improvement is fundamentally a *decidability-bounded* process. Its effectiveness is governed not by generation power, compute, or intelligence, but by the computability class of the self-verification predicate. Where verification is decidable and tractable, RSI compounds (Theorem 4.3(1)). Where it is **NP**-hard, RSI stalls (Theorem 4.3(2)). Where it is undecidable—the general case for expressive systems—RSI either hallucinates, stalls, or requires external grounding (Theorem 5.5, Corollary 6.10).

The **P** versus **NP** question captures the generation-verification asymmetry at the resource level. The halting problem captures it at the level of existence (Theorem 3.5). The latter subsumes the former. The correct object of study is a *self-verification class* whose properties remain open.

The field has treated RSI as a capability to be scaled. It is not. It is a structure to be satisfied, and the structure is the same one Gödel, Turing, and Rice identified. The surprise is not that RSI is hard. The surprise is that hardness was never the right frame.

Intelligence is a quantity. Self-improvement is a structure. The structure is undecidable, and no quantity of intelligence changes a structure.

Fast takeoff is not ruled out. But it requires a system to solve, in general, a problem that three of the deepest results in mathematical logic say cannot be solved in general. The burden of proof is on the takeoff hypothesis.

We began by observing that the RSI debate has been a debate about rate. We end by observing that rate is downstream of structure, and structure is downstream of decidability. The debate, correctly framed, has not yet begun.

A Notation

Symbol	Meaning
$\mathcal{M}$	Class of computable systems
$\text{cap}(M)$	Capability measure (semantic, non-trivial)
$\preceq$	Capability partial order
$\mathcal{G}$	Generation operator
$\mathcal{V}$	Verification predicate
$\mathcal{V}^*$	Ideal verifier
$\mathcal{R}$	Effective RSI step
$\mathcal{A}$	Adoption rule
$\Pi_{\mathcal{V}}$	Decision problem induced by $\mathcal{V}$
O	External oracle
K	Halting problem
$\leq_T$	Turing reducibility
hal	Hallucination rate

Table 2: Notation.

B Relation to Prior Work

- **Gödel (1931)** — incompleteness; the first barrier to self-verification.
- **Turing (1936)** — halting problem; the barrier to self-decision.
- **Rice (1953)** — undecidability of semantic properties; the barrier to improvement verification.
- **Good (1966)** — meta-utility and self-modifying agents; the fixed-point problem in embryo.
- **Schmidhuber (2003)** — Gödel machines; verification assumed sound by construction, which our results show is unavailable in general.
- **Yudkowsky (2008)** — intelligence explosion; we supply the computability-theoretic bound.
- **Shumailov et al. (2023)** — model collapse under self-generated data; a special case of Section 5.
- **AlphaZero, STaR, self-play** — empirical instances of decidable-regime RSI, per the taxonomy of Table 1.

C Falsifiability

The framework is falsifiable. It predicts:

1. RSI systems in the undecidable regime will exhibit hallucination rates that *do not decrease* with scale or self-critique, absent external grounding.
2. Self-training loops in open-ended domains will either *collapse* (drift) or *stall* (sound-incomplete), never compound unboundedly.
3. Systems with decidable verifiers (proof, code, games) will show *compounding* improvement; systems without will not.
4. Any claimed general RSI system will, on inspection, be found to rely on an *external oracle* (human feedback, retrieval, environment).

A demonstration of unbounded general RSI without external grounding would falsify the framework.

This paper is a formalization of the thesis that recursive self-improvement is bounded by decidability, not intelligence. The central claim—that general RSI is undecidable by default and requires grounding—is stated to be falsifiable and is offered as the correct frame for evaluating takeoff hypotheses.